

Unit-III

Instruction-Level Parallelism: Concepts and Challenges

UNIT - III

- Advanced Architecture Features
- Instruction-Level Parallelism: Concepts and Challenges-Basic Compiler Techniques for Exposing ILP
- Reducing Branch Costs with Advanced Branch Prediction
- Overcoming Data Hazards with Dynamic Scheduling
- Dynamic Scheduling: Examples and the Algorithm
- Hardware-Based Speculation

Introduction

- All processors since about 1985 have used pipelining to overlap the execution of instructions and improve performance.
- This **potential overlap among instructions is called instruction-level parallelism (ILP)**, because the instructions can be evaluated in parallel.
- Approaches to Exploiting ILP
 - Dynamic Scheduling (Hardware): The processor detects and handles dependencies at runtime (e.g., Tomasulo's Algorithm, Scoreboarding). This is typical in modern CPUs.
 - Static Scheduling (Compiler): The compiler analyzes the code, rearranges it, and fills in gaps (e.g., in VLIW processors) at compile-time to avoid hazards.

Processors using the **dynamic, hardware-based approach, including all recent Intel and many ARM processors**, dominate in the desktop and server markets

Parameter	Static Scheduling	Dynamic Scheduling
Definition	Scheduling decided before execution Ex: I1: $R1 = R2 + R3$ I2: $R4 = R1 + R5$ (depends on I1) I3: $R6 = R7 + R8$ (independent) Compiler rearranges: I1 I3 ← moved to reduce delay I2	Scheduling decided during execution Ex: I1: $R1 = R2 + R3$ I2: $R4 = R1 + R5$ I3: $R6 = R7 + R8$ CPU executes: I1 I3 ← executed before I2 automatically I2
Done by	Compiler	Hardware (CPU)
Time of scheduling	Compile-time	Run-time
Instruction order	Fixed before execution	Can change dynamically (out-of-order execution)
Dependency handling	Analyzed by compiler	Detected and handled by hardware
Flexibility	Low	High
Adaptability	Cannot adapt to runtime changes	Adapts to runtime conditions
Hardware complexity	Simple	Complex
Compiler complexity	High	Moderate
Performance	Limited (depends on compile-time decisions)	High (better resource utilization)
Handling delays	Cannot handle unexpected delays	Handles delays (cache miss, memory latency)
Examples	VLIW processors	Tomasulo's Algorithm, Scoreboarding
Power consumption	Low	High

Key concepts in ILP

- **Pipelining:** Overlapping the stages of instruction execution (fetch, decode, execute) like an assembly line.
- **Superscalar Execution:** A processor with multiple functional units (ALUs, FP units) fetches, decodes, and executes multiple instructions in parallel.
- **Out-of-Order (OoO) Execution:** The hardware rearranges instructions at runtime to execute them as soon as their required operands are available, rather than in the original program order.
- **Register Renaming:** Hardware eliminates false dependencies by mapping architectural registers to a larger set of physical registers, preventing unnecessary serialization.
- **Branch Prediction & Speculation:** To keep the pipeline full, the processor predicts the path of a branch (if/else) and executes instructions before knowing for sure if they are required. If correct, performance increases; if wrong, the results are discarded.
- **Loop Unrolling:** A compiler technique that replicates the body of a loop multiple times, reducing branch overhead and increasing the pool of independent instructions.

- The value of the CPI (cycles per instruction) for a pipelined processor is the sum of the base CPI and all contributions from stalls:

$$\text{Pipeline CPI} = \text{Ideal pipeline CPI} + \text{Structural stalls} + \text{Data hazard stalls} + \text{Control stalls}$$

- The ideal pipeline CPI is a measure of the maximum performance attainable by the implementation

Data Dependences and Hazards

- To exploit instruction-level parallelism, determine which instructions can be executed in parallel.
- If two instructions are parallel, they can execute simultaneously in a pipeline without causing any stalls.
- If two instructions are dependent they are not parallel and must be executed in order.
- There are three different types of dependences:
 - (i) data dependences
(also called true data dependences),
 - (ii) name dependences, and
 - (iii) control dependences.
- The key in both cases is to determine whether an instruction is dependent on another instruction

Data Dependences

- An instruction j is data-dependent on instruction i if either of the following holds:
 - Instruction i produces a result that may be used by instruction j .
 - Instruction j is data-dependent on instruction k , and instruction k is data dependent on instruction i .

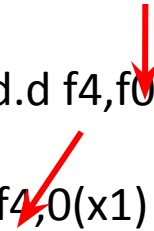
Data Dependences

- For example, consider the following RISC-V code sequence that increments a vector of values in memory (starting at 0(x1) ending with the last element at 0(x2)) by a scalar in register f2.

```
Loop: fld f0,0(x1)      //f0=array element
      fadd.d f4,f0,f2    //add scalar in f2
      fsd f4,0(x1)       //store result
      addi x1,x1,-8      //decrement pointer 8 bytes
      bne x1,x2,Loop     //branch x1 is not equal x2
```

The data dependences in this code sequence involve both floating-point data:

```
Loop: fld f0,0(x1)      //f0=array element
      fadd.d f4,f0,f2    //add scalar in f2
      fsd f4,0(x1)       //store result
```



and integer data:

```
      addi x1,x1,-8      //decrement pointer //8 bytes (per DW)
      bne x1,x2,Loop     //branch x1ax2
```



- A data dependence conveys three things:
 - (1) the possibility of a hazard,
 - (2) the order in which results must be calculated, and
 - (3) an upper bound on how much parallelism can possibly be exploited

(ii) Name Dependences

- A name dependence occurs when two instructions use the same register or memory location, called a name, but there is no flow of data between the instructions associated with that name.
- There are two types of name dependences between an instruction *i* that precedes instruction *j* in program order:
 - An **antidependence** between instruction *i* and instruction *j* occurs when instruction *j* writes a register or memory location that instruction *i* reads. The original ordering must be preserved to ensure that *i* reads the correct value. In the following example, there is an antidependence between `fsd` and `addi` on register `x1`.

<code>fsd f4,0(x1)</code>	<code>//store result</code>
<code>addi x1,x1,8</code>	<code>//decrement pointer 8 bytes</code>
 - An **output dependence** occurs when instruction *i* and instruction *j* write the same register or memory location. The ordering between the instructions must be preserved to ensure that the value finally written corresponds to instruction *j*.

Data Hazards

- Consider two instructions i and j , with i preceding j in program order. The possible data hazards are

RAW (read after write):

1. $i = i + 1$;

2. $j = i + 1$

- j tries to read a source before i writes it, so j incorrectly gets the old value.
- This hazard is the most common type and corresponds to a true data dependence.

WAW (write after write):

1. $i = i + 1$;

2. print i ;

3. $i = j + 1$

- j tries to write an operand before it is written by i —this corresponds to the output dependence;

WAR (write after read):

1. read i ;

2. $i = j + 1$

- j tries to write a destination before i has read it—this corresponds to an anti-dependency.

Hazard Type	Full Form	Dependency Type	Problem
RAW	Read After Write	True dependency	Reads old value
WAW	Write After Write	Output dependency	Wrong write order
WAR	Write After Read	Anti dependency	Reads incorrect (new) value

RAW Hazard (Read After Write)

- A RAW hazard occurs when an instruction attempts to read a data value before the previous instruction has finished writing it.
- This creates a dependency, as the reading instruction cannot proceed until the writing instruction completes.

```
ADD R1, R2, R3 ; (R2 & R3 are the  
read registers, R1  
is write  
register)
```

WAW Hazard (Write After Write)

- A WAW hazard occurs when two consecutive instructions attempt to write to the same destination register .
- The pipeline doesn't know which write operation should take effect, creating ambiguity.

ADD R1, R2, R3 ; Write to R1
in cycle 2 (Execute stage)

MUL R1, R4, R5 ; Write to R1
in cycle 3 (Decode

WAR Hazard (Write After Read)

- A WAR hazard occurs when an instruction attempts to write to a data value after a previous instruction has already read it.
- The pipeline needs to ensure the reading instruction uses the correct value before the writing instruction modifies it.

LOAD R1, Mem[100] ; Read
from memory location 100 to R1
in cycle 2 (Execute stage)

ADD R2, R1, R3 ; Use the

(iii) Control Dependences

- The last type of dependence is a control dependence. A control dependence determines the ordering of an instruction, *i*, with respect to a branch instruction so that instruction *i* is executed in correct program order and only when it should be.

```
if p1
  { S1; };
if p2
  { S2; }
```

S1 is control-dependent on p1, and

S2 is control-dependent on p2 but not on p1.

In general, two constraints are imposed by control dependences:

1. An instruction that is control-dependent on a branch cannot be moved before the branch so that its execution is no longer controlled by the branch. For example, we cannot take an instruction from the then portion of an if statement and move it before the if statement.
2. An instruction that is not control-dependent on a branch cannot be moved after the branch so that its execution is controlled by the branch. For example, we cannot take a statement before the if statement and move it into the then portion.

Technique	Reduces
Forwarding and bypassing	Potential data hazard stalls
Simple branch scheduling and prediction	Control hazard stalls
Basic compiler pipeline scheduling	Data hazard stalls
Basic dynamic scheduling (scoreboarding)	Data hazard stalls from true dependences
Loop unrolling	Control hazard stalls
Advanced branch prediction	Control stalls
Dynamic scheduling with renaming	Stalls from data hazards, output dependences, and antidependences
Hardware speculation	Data hazard and control hazard stalls
Dynamic memory disambiguation	Data hazard stalls with memory
Issuing multiple instructions per cycle	Ideal CPI
Compiler dependence analysis, software pipelining, trace scheduling	Ideal CPI, data hazard stalls
Hardware support for compiler speculation	Ideal CPI, data hazard stalls, branch hazard stalls

Implementation of ILP and overcoming hazards or dependencies.

- To implement ILP, 3 methods are used to avoid any delay during ILP
 1. score boarding.
 2. Tomasulo's solution for dynamic scheduling.
 3. Branch prediction.

What Is Instruction-Level Parallelism?

- **Basic Block:** The amount of parallelism available within a **basic block**—a straight-line code sequence with no branches in except to the entry and no branches out except at the exit—is quite small.
- The simplest and most common way to increase the ILP is to exploit parallelism among iterations of a loop. This type of parallelism is often called loop-level parallelism.

```
w = 0;  
x = x + y;  
y = 0;  
if( x > z )  
{  
    y = x;  
    x++;  
}  
else  
{  
    y = z;  
    z++;  
}  
w = x + z;
```

Source Code

```
w = 0;  
x = x + y;  
y = 0;  
if( x > z )
```

```
y = x;  
x++;
```

```
y = z;  
z++;
```

```
w = x + z;
```

Basic Blocks